

Hash

一、概述

Hash，一般翻译做散列、杂凑，或音译为哈希，是把任意长度的输入（又叫做预映射 pre-image）通过散列算法变换成固定长度的输出，该输出就是散列值。这种转换是一种压缩映射，也就是说，散列值所占空间通常远小于输入的空间，不同的输入可能会散列成相同的输出，所以不可能通过散列值来确定唯一的输入值。简单的说 Hash 就是一种将任意长度的消息压缩到某一固定长度的消息摘要的函数。

Hash 算法可以将一个数据转换为一个**消息摘要/标志/散列值**，这个标志和源数据的每一字节内容都有十分紧密的关系。Hash 算法还具有一个特点，就是很难找到逆向规律。

Hash 算法是一个广义的算法，也可以认为是一种思想，使用 Hash 算法可以提高存储空间的利用率，可以提高数据的查询效率，也可以做数字签名来保障数据传递的安全性。所以 Hash 算法被广泛地应用在互联网应用中。

举个例子：假设有一集合 S，里面装满了形如<key, value>的键值对，key 值是唯一的。如果想在尽可能短的时间内取出 key=x 对应的 value 的值，应当如何做？

处理这个问题首先有几个难点：若是直接根据 key 值存储 value 的值 $a[key]=value$ ，存在 key 值变化范围大、数组下标无法驾驭的情况；若是分别存储 key 值与 value 值，存在需要穷举查找符合要求的 key 值，在时间上又存在困难。

假设集合中存在 N 个不同的 key，那么当我们可以定义一个函数以 key 为自变量的函数 F，它能使得 $F(key)$ 等于某个形如 $[1, N]$ 的整数，并且对于 $\forall key1 \neq key2$ ，可能存在 $F(key1) \neq F(key2)$ 。当计算 F() 的代价是 $O(1)$ 时，我们可以通过开一个大小为 N 的类似桶的存储 value 类型的值的数组就可以实现存储和取值了，其时间复杂度和计算 F() 等价。我们也把这个数组称作**哈希表**。

示例： $hash[F(key)]=value$

如上，那么 F() 函数则被称为哈希函数。哈希函数也可叫做哈希算法，它可以用于检验信息是否相同(文件检验)，或者检验信息的拥有者是否真实(数字签名)。

二、哈希的构造方法

构造哈希函数的方法有很多。在介绍各种方法前，首先需要明确什么是“好”的哈希算法。若对于关键字集中的任一个关键字，经哈希函数映像到地址集中任何一个地址的概率是相等的，则称此类哈希函数是均匀的（Uniform）哈希函数。换句话说，就是使关键字经过哈希函数得到一个“随机的地址”，以便使一组关键字的哈希地址均匀分布在整个地址区间中，从而减少冲突。

1.直接定址法

取关键字或关键字的某个线性函数值为哈希地址。即：

$$H(key) = key \text{ 或 } H(key) = a * key + b$$

其中 a 和 b 为常数（这种哈希函数叫做自身函数）。

例如：有一个 1 岁到 100 岁的人口数字统计表，其中，年龄作为关键字，哈希函数取关键字自身。如下表所示($H(key)=1*key+0$)：

地址	01	02	03	...	25	26	27	...	100
年龄	1	2	3	...	25	26	27	...	100
人数	3000	2000	5000	...	1050	...	...	...	...

这样若要询问 25 岁的人有多少，则只要查表的第 25 项即可。

由于直接定址所得的地址集合和关键字集合的大小相同。因此，对于不同的关键字不会发生冲突，但是实际中能使用这种哈希函数的情况很少。

2. 数字分析法

假设关键字是以 r 为基数的数（如：以 10 为基的十进制数），并且哈希表中可能出现的关键字都是事先知道的，则可取关键字的若干数位组成哈希地址。

例如有 80 个记录，其关键字为 8 位十进制数。假设哈希表的表长为 $(100)_{10}$ ，则可取两位十进制数组成哈希地址。那么取哪两位呢？原则是使用得到的哈希地址尽量避免产生冲突，则需从分析这 80 个关键字着手。假设这 80 个关键字中的一部分如下所列：

.....							
8	1	3	4	6	5	3	2
8	1	3	7	2	2	4	2
8	1	3	8	7	4	2	2
8	1	3	0	1	3	6	7
8	1	3	2	2	8	1	7
8	1	3	3	8	9	6	7
8	1	3	5	4	1	5	7
8	1	3	6	8	5	3	7
8	1	4	1	9	3	5	5
.....							
①	②	③	④	⑤	⑥	⑦	⑧

对关键字的全体分析中我们发现：第①②位都是“81”，第③位只可能取值 1、2/3 或 4，第⑧位只可能取值 2、5 或 7，因此这 4 位都不可取。由于中间的 4 位都可看成是近乎随机的，因此可取其中任意两位，或取其中两位与另两位的叠加求和后舍去进位作为哈希地址。

3. 平方取中法

取关键字平方后的中间几位为哈希地址。这是一种较常用的构造哈希函数的方法。通常在选定哈希函数时不一定能知道关键字的全部情况，取其中哪几位也不一定合适，而一个数平方后的中间几位数和数的每一位都有关，由此使随机分布的关键字得到的哈希地址也是随机的。取的位数由表长决定。

4. 折叠法

将关键字分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去进位）作为哈希地址，这种方法称为折叠法（folding）。关键字位数很多，而且关键字中每一位数字分布大致均匀时，可以采用折叠法得到哈希地址。

例如：每一中西文图书都有一个国际标准图书编号（ISBN），它是一个 10 位的十进制

数字，若要以它作关键字建立一个哈希表，当馆藏书种类不到 10000 时，可采用折叠法构造一个四位数的哈希函数。在折叠法中数位叠加可以有**移位叠加**和**间界叠加**两种方法。移位叠加是将分割后的每一部分的最低位对齐，然后相加；间界叠加是从一端向另一端沿分割界来回折叠，然后对齐相加。如国际标准图书编号 O-442-20586-4 的哈希地址分别如下所示：

移位叠加：	间界叠加：
5864+	5864+
4220+	0224+
04=	04=
10088	6092
H(key)=0088	H(key)=6092

5.除留余数法

取关键字被某个不大于哈希表表长 m 的数 p 除后所得的余数为哈希地址。即

$$H(key) = key \text{ MOD } p, p \leq m$$

这是一种**最简单，也最常用的**构造哈希函数的方法。它不仅可以对关键字直接取模（MOD），也可以在折叠、平方取中等运算之后取模。

值得注意的是，在使用除留余数法时，对 p 的选择很重要。若 p 选的不好，容易产生相同的散列值。

例如，已知散列元素为（18、75、60、43、54、90、46），表长 $m=10$ ， $p=7$ ，则有

$$\begin{aligned} h(18) &= 18 \% 7 = 4, & h(75) &= 75 \% 7 = 5, & h(60) &= 60 \% 7 = 4, \\ h(43) &= 43 \% 7 = 1, & h(54) &= 54 \% 7 = 5, & h(90) &= 90 \% 7 = 6, \\ h(46) &= 46 \% 7 = 4 \end{aligned}$$

由于冲突较多，为减少冲突，可取较大的 m 值和 p 值，如 $m=p=13$ ，结果如下：

$$\begin{aligned} h(18) &= 18 \% 13 = 5, & h(75) &= 75 \% 13 = 10, & h(60) &= 60 \% 13 = 8, \\ h(43) &= 43 \% 13 = 4, & h(54) &= 54 \% 13 = 2, & h(90) &= 90 \% 13 = 12, \\ h(46) &= 46 \% 13 = 7 \end{aligned}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
		54		43	18		46	60		75		90

理论研究表明，除留余数法的模 p 取不大于表长且最接近表长 m 的素数效果最好，且 p 最好取 $1.1^n \sim 1.7^n$ 之间的一个素数（ n 为存在的数据元素个数）。

6.随机数法

选择一个随机函数，取关键字的随机函数值为它的哈希地址，即 $H(key) = \text{random}(key)$ ，其中 random 为随机函数。通常，当关键字长度不等时采用此法构造哈希函数较恰当。

以上便是常用的几种构造哈希函数的方法，实际工作中需视不同的情况采用不同的哈希函数，或者说，自行设计适合当前情境的哈希函数。通常考虑的因素有：

计算哈希函数所需时间（包括硬件指令的因素）。

关键字的长度。

哈希表的大小。

关键字的分布情况。

记录的查找频率。

而在 OI/ACM 最常用到的 hash 函数，我们一般称为：进制哈希。

以字符串 “Reimu” 为例，我们将使用 hash 函数计算其散列值，一种方法为：

$F(\text{“ABC”}) = ((1 \times 26 + 2) \times 26) + 3$

或 $F(\text{“ABC”}) = (65 \times 128 + 66) \times 128 + 67$

或

鉴于进制转换出来的散列值会偏大，所以往往配合除留余数法：

如 $F(\text{“ABC”}) = (((1 \times 26 + 2) \times 26) + 3) \% 99997$

或 $F(\text{“ABC”}) = ((65 \times 128 + 66) \times 128 + 67) \% 123456987$

或

基数的选取与除数的选取可自由变换，通常要根据具体情况进行设计，一般来说，建议除数为**大质数**。（原因请自行查询 [生日悖论](#)）

以洛谷 P3370 【模板】 字符串哈希为例：

题目描述

如题，给定 N 个字符串（第 i 个字符串长度为 M_i ，字符串内包含数字、大小写字母，大小写敏感），请求出 N 个字符串中共有多少个不同的字符串。

#友情提醒：如果真的想好好练习哈希的话，请自觉，否则请右转 PJ 试炼场:)

输入格式

第一行包含一个整数 N ，为字符串的个数。

接下来 N 行每行包含一个字符串，为所提供的字符串。

输出格式

输出包含一行，包含一个整数，为不同的字符串个数。

输入输出样例

输入

```
5
abc
aaaa
abc
abcc
12345
```

输出

```
4
```

说明/提示

时空限制：1000ms,128M

数据规模：

对于 30% 的数据： $N \leq 10$, $M_i \leq 6$, $M_{\max} \leq 15$;

对于 70% 的数据： $N \leq 1000$, $M_i \leq 100$, $M_{\max} \leq 150$

对于 100% 的数据： $N \leq 10000$, $M_i \leq 1000$, $M_{\max} \leq 1500$

样例说明：

样例中第一个字符串(abc)和第三个字符串(abc)是一样的，所以所提供字符串的集合为 {aaaa,abc,abcc,12345}，故共计 4 个不同的字符串。

三、哈希的冲突处理

在实际使用中，有个不可避免的问题，就是**哈希冲突/碰撞**。所谓哈希冲突，就是对于特定的哈希函数，形如“Satori”和“5”的哈希值相同的情况。例如上面提到的字符串进制哈希，若存在两字符串在不取模 p 的情况下所得的进制总和为 $p*z+x$ 与 $p*y+x$ ，则在最终取模之后两个字符串的哈希值是**相同/冲突**的。正如前面提到的，**均匀的哈希函数可以减少冲突，但不能避免**。因此，如何处理冲突是构造哈希表时不可缺少的一步。

通常用的处理冲突的方法有下列几种：

1.开放定址法

$$H_i = (H(\text{key}) + d_i) \text{ MOD } m \quad i = 1, 2, \dots, k \quad (k \leq m-1)$$

其中： $H(\text{key})$ 为哈希函数； m 为哈希表表长； d_i 为增量序列，可有下列 3 种取法：

- (1). $d_i = 1, 2, 3, \dots, m-1$, 称**线性探测再散列**。
- (2). $d_i = 1^2, -1^2, 2^2, -2^2, \dots, \pm k^2 \quad (k \leq m/2)$ 称**二次探测再散列**。
- (3). d_i = 伪随机数序列, 称**伪随机探测再散列**。

例如，在长度为 11 的哈希表中已填有关键字分别为 17, 60, 29 的记录（哈希函数 $H(\text{key}) = \text{key} \text{ MOD } 11$ ），现有第四个记录，其关键字为 38，由哈希函数得到哈希地址为 5，产生冲突。若用线性探测再散列方法处理，得到下一个地址为 6，仍冲突，继续算 7，仍冲突，知道最后算出 8，填入哈希表。若用二次探测再散列，则应填入需要为 4 的位置。伪随机数序列则为生成伪随机地址再散列。

0	1	2	3	4	5	6	7	8	9	10
					60	17	29			
插入前										
					60	17	29	38		
线性探测再散列										
				38	60	17	29			
二次探测再散列										
			38		60	17	29			
伪随机探测再散列										

2.再哈希法

$$H_i = RH_i(\text{key}) \quad i = 1, 2, \dots, k$$

RH_i 均是不同的哈希函数，即在同义词产生地址冲突时计算另一个哈希函数地址，直到冲突不再发生。这个方法不易产生“聚集”，但增加了计算的时间。

3.链地址法

将所有关键字为同义词的记录存储在同一线性链表中。假设某哈希函数产生的哈希地址在区间 $[0, m-1]$ 上，则设立一个指针型向量

Chain ChainHash[m]

其每个分量的初始状态都是空指针。凡哈希地址为 i 的记录都插入到头指针为 ChainHash[i] 的链表中。在链表中的插入位置可以在表头或表尾；也可以在中间，以保持同

义词在同一线性链表中按关键字有序。

4. 建立一个公共溢出区

这也是处理冲突的一种方法、假设哈希函数的值域为 $[0, m-1]$ ，则设向量 $\text{HashTable}[0..m-1]$ 为基本表，每个分量存放一个记录，另设向量 $\text{OverTable}[0..v]$ 为溢出表。所有关键字和基本表中关键字为同义词的记录，不管它们由哈希函数得到的哈希地址是什么，一旦发生冲突，都填入溢出表。

在 OI/ACM 的环境下，对于 Hash 冲突的避免我们往往采用扩大模数的方式，常用的有自然溢出、多重哈希等。

自然溢出

由于模数 P 的选取往往选择大质数，而在比赛中记忆大质数是比较困难的事情。所以退而求其次，自动除以一个较大的模数，例如，利用数据类型自然溢出的特性，设定为 `unsigned long long` 类型，不手动进行取模，在数值溢出时会自动对 2^{64} 进行取模，这种方法会不会被卡取决于出题人的良心。如 BZOJ 3097 Hash Killer I。

多重哈希

由于记不得一个大质数，那可以记忆两个相对较大的质数，通过对两个质数取模产生两个哈希值。当且仅当两个哈希值都相同时才会产生冲突的可能，这种可能当且仅当两未取模的哈希值为 $y \cdot p_1 \cdot p_2 + x$ 和 $z \cdot p_1 \cdot p_2 + x$ 的情况。当 p_1 、 p_2 足够大时，实际模拟了两两最大公约数 $p = p_1 \cdot p_2$ 的效果。

通过自然溢出、多重哈希等方法求得的哈希值可以直接标志一个字符串/大数据串。也就是说，我们可以通过比较哈希值，来直接比较两者是否相同。这个方法同样适用于比较两字符串的子串是否相同。

对于子串的处理

用 $\text{hash}[r]$ 记录字符串进制哈希 $s[1..r]$ 的哈希值，进制基数为 k ，则子串 $s[l..r]$ 的哈希值为：

$$\text{hash}[r] - \text{hash}[l-1] \cdot k^{r-l+1}$$

我们知道，进行进制哈希的过程本质上就是把原先得到的哈希值在基数进制上强行左移一位，然后放进去当前的这个字符。

现在的目的是，取出 l 到 r 这段子串的 hash 值，也就是说 $h[l-1]$ 这一段是没有用的，我们把在 $h[r]$ 这一位上， $h[l-1]$ 这堆字符串的 hash 值做的左移运算全部还原给 $h[l-1]$ ，就可以知道 $h[l-1]$ 在 $h[r]$ 中的 hash 值，那么减去即可。（简单的容斥思想）

四、哈希表的查询与分析

在哈希表上进行查找的过程和哈希建表的过程基本一致。给定 K 值，根据建表时设定的哈希函数求得哈希地址，若表中此位置上没有记录，则查找不成功；否则比较关键字，若和给定值相等，则查找成功；否则根据造表时设定的处理冲突的方案找“下一地址”，直到找到为止。

在 OI/ACM 比赛中，有时候不仅仅需要哈希值，还需要配合哈希表对键值对 $\langle \text{key}, \text{value} \rangle$ 进行一定的分析和处理，此时哈希值还具有哈希表中“地址”的意义，存储中产生冲突/碰撞就需要一定的处理方式。

常用的方法参见线性探测再散列（Linear Probe Hashing）、再哈希法（双重哈希）、链地址法。

使用 hash 的几个要注意的地方

在复杂度允许的情况下，尽量采用多重 hash（不过一般双重 hash 就够）

比赛时能不用自然溢出就不要（平时刷题如果用自然溢出被卡可以及时换掉，但是比赛时如果用自然溢出，OI 赛制就 GG 了）

模数用大质数这个不用说了。

并且进制基数不要选太简单的，比如 233 和 13131 这样的，尽量大一点，比如 13131131 和 233333。太小容易被卡。

以及要合理应对各种卡 hash 方法的最好方法就是自己去卡一遍 hash。

【拓展练习】：

Hdu1686、Poj2774、Poj3261、UVA-257、UVA-11019、Hdu4821、Hdu2859、CF RC2018 R3 E、CF 727E……

墙裂推荐 BZOJ(衡阳大视野, lydsy.com, 但是已经没了)的 hash killer 系列，对于了解怎么（被）卡 hash 很有帮助。

参考内容：

<https://www.jianshu.com/p/4cc2cc287e9f>

<https://zhuanlan.zhihu.com/p/89381173>

<https://www.nowcoder.com/discuss/178326?type=101>

https://blog.csdn.net/qq_38891827/article/details/80723483

整理编辑：绍兴市第一中学 Kcm